

实验二 原生应用开发总结报告

一、项目概述

基于实验二的要求，在 Xcode 上使用 SwiftUI 实现了完整的登录功能模块，包括登录页面、个人中心界面和隐私保护功能。

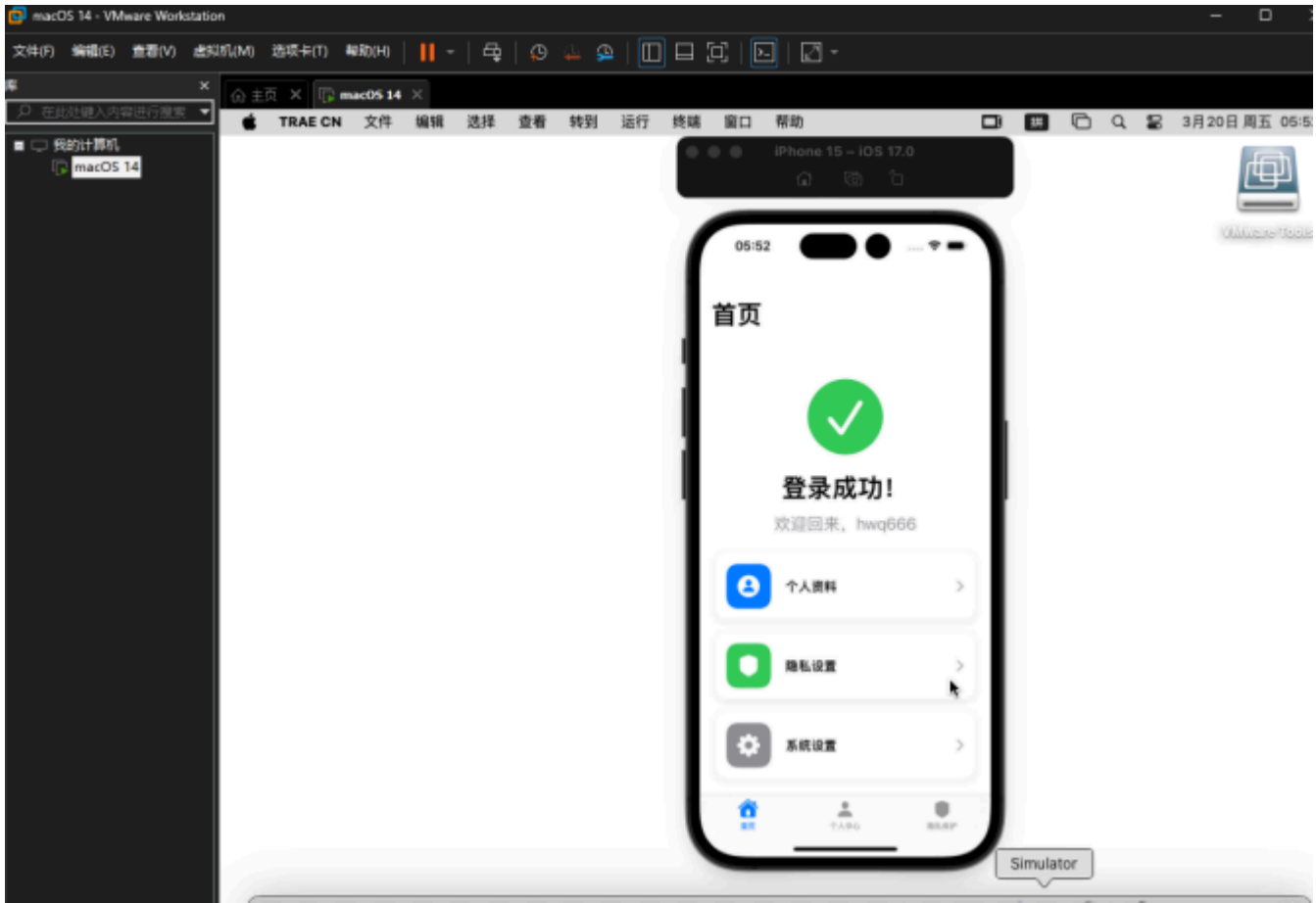
二、功能实现

1. 登录功能

文件: LoginView.swift

功能项	实现方式
用户名输入	<code>TextField</code> + 输入验证
密码输入	<code>SecureField</code> 安全输入
输入验证	空值检测、凭证验证
错误提示	红色文字动态显示
验证凭证	用户名: <code>hwq666</code> , 密码: <code>2023210596</code>

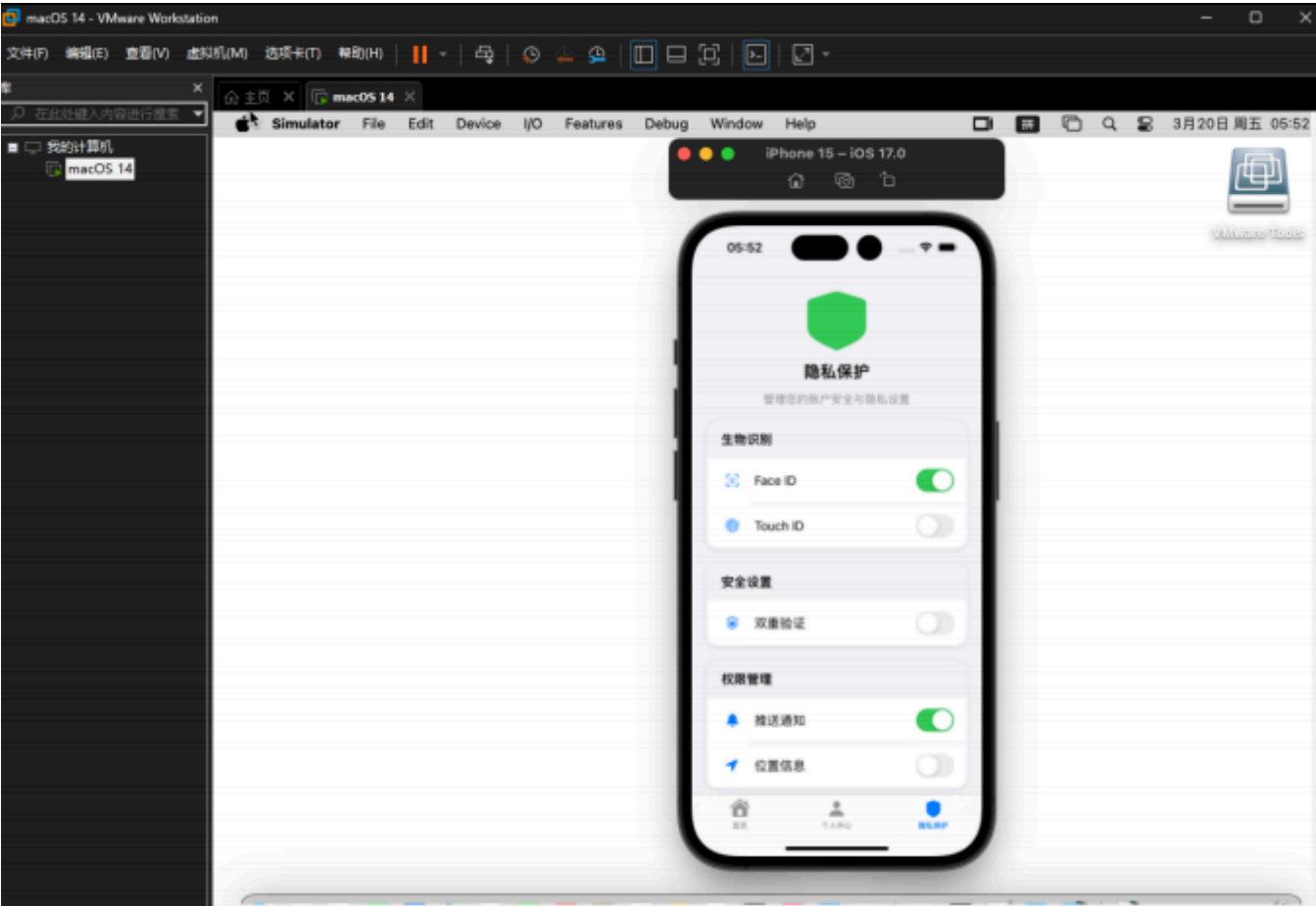
2. 个人中心界面



文件: ProfileView.swift

功能项	实现方式
用户头像	<code>Image(systemName: "person.circle.fill")</code>
用户信息展示	卡片式列表布局
信息项	姓名、学号、邮箱、学院、专业
数据传递	通过 <code>@Binding</code> 接收用户名

3. 隐私保护功能



文件: PrivacyView.swift

功能项	实现方式
生物识别	Face ID / Touch ID 开关
安全设置	双重验证开关
权限管理	推送通知、位置信息开关
修改密码	功能入口按钮

4. 主界面架构

文件: MainView.swift

使用 `Tabview` 实现底部导航栏，集成三个页面：

- 首页
- 个人中心
- 隐私保护

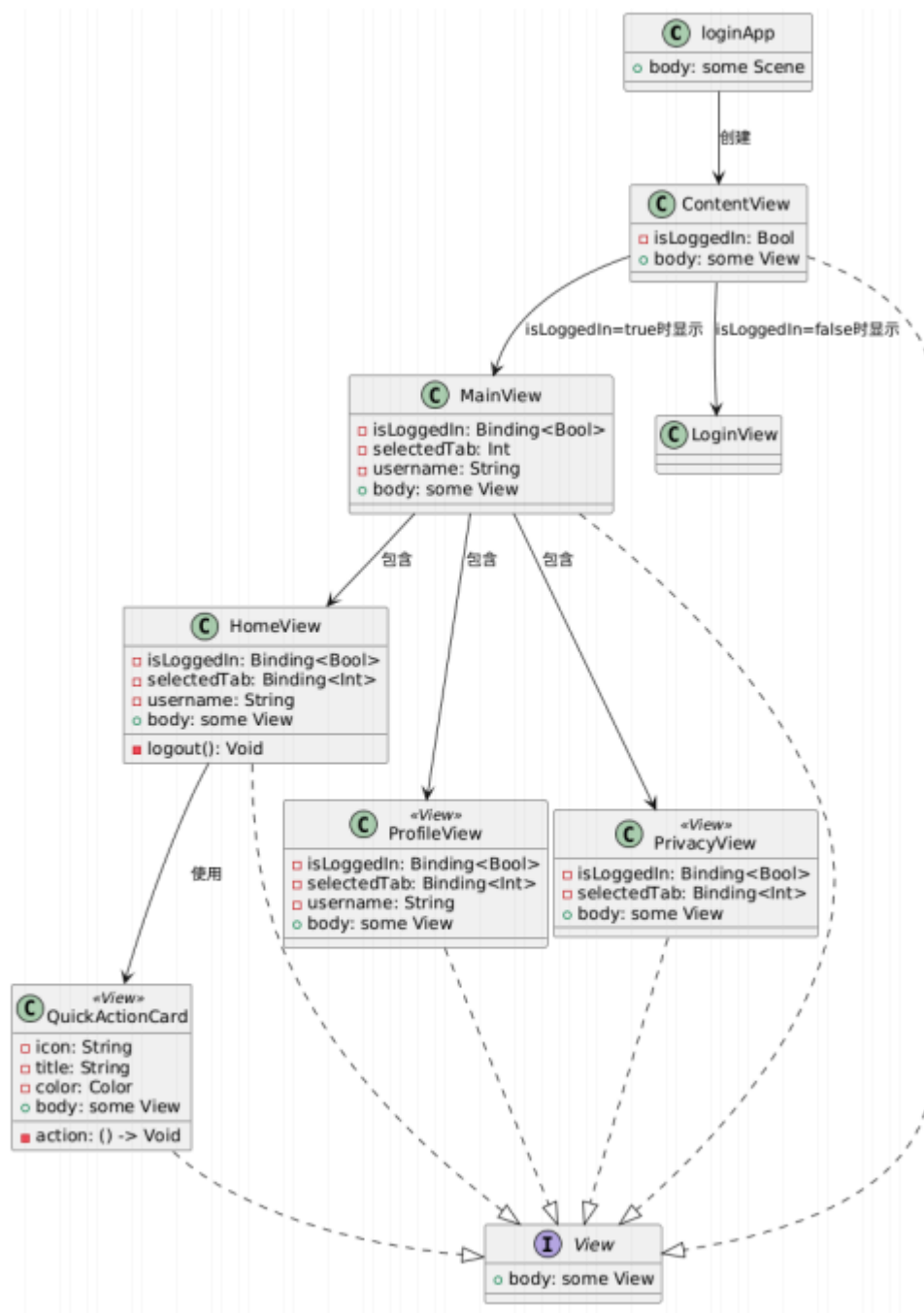


图4.1 类图

三、技术要点

SwiftUI 核心组件使用

组件	用途
@State	管理视图内部状态
@Binding	父子视图间双向数据绑定
TabView	底部标签导航
NavigationView	导航栏管理
ScrollView	可滚动内容区域
Toggle	开关控件

数据传递方式

ContentView (isLoggedIn)
├─ LoginView (@Binding isLoggedIn)
├─ MainView (@Binding isLoggedIn)
│ ├─ HomeView (@Binding isLoggedIn, @Binding selectedTab, username)
│ ├─ ProfileView (@Binding isLoggedIn, @Binding selectedTab, username)
│ └─ PrivacyView (@Binding isLoggedIn, @Binding selectedTab)

四、平台差异对比

根据实验要求，以下是 iOS 与 Android 开发的对比：

对比项	iOS (SwiftUI)	Android (Kotlin)
开发语言	Swift	Kotlin
UI框架	SwiftUI 声明式	XML/Compose
页面跳转	NavigationView/TabView	Intent
数据传递	@Binding/@State	Intent.putExtra
输入框	TextField/SecureField	EditText
密码输入	SecureField	inputType="textPassword"
列表布局	VStack + ForEach	RecyclerView
开发工具	Xcode	Android Studio

五、项目文件结构

```
login/
├─ loginApp.swift          # 应用入口
├─ ContentView.swift      # 根视图（登录状态管理）
├─ LoginView.swift        # 登录页面
├─ MainView.swift         # 主界面
├─ ProfileView.swift      # 个人中心
└─ PrivacyView.swift      # 隐私保护
```

六、实验成果

- ✅ iOS 登录页面功能完整
- ✅ 输入验证逻辑正确
- ✅ 个人中心界面实现
- ✅ 隐私保护功能实现
- ✅ TabView 导航正常
- ✅ 数据传递正常

七、心得体会

- SwiftUI 声明式语法的优势**：代码简洁，易于理解和维护
- @Binding 的使用**：实现了父子组件间的数据共享，避免了复杂的状态管理
- 组件化开发**：将不同功能拆分为独立的 View，提高了代码复用性
- AI 辅助开发**：使用 TRAE 快速生成代码骨架，提高了开发效率